

TP1 - Resolución de problemas con algoritmos.

Materia: Programación II

Objetivo: Escribir los primeros programas en C++ a partir de un problema concreto.

Introducción

Para cada uno de los siguientes ejercicios, se solicita:

1. **Identificar las variables** de entrada y salida y escribir las precondiciones y postcondiciones.
 2. **Definir el modelo matemático** (ecuaciones, inecuaciones o lógica de conjuntos).
 3. **Diseñar el algoritmo** en C++ utilizando funciones y estructuras de control.
-

1. El Costo de la Infraestructura

Problema: Una empresa debe conectar dos nodos de red en un terreno rectangular de X metros de ancho por Y de largo. Los nodos están en esquinas opuestas. El cableado por el borde del terreno cuesta \$10 el metro, pero cruzarlo en diagonal (subterráneo) cuesta \$18 el metro. Crear una función que reciba X e Y y determine qué opción es más económica.

2. Trayectoria de Projectiles (Balística)

Problema: Un dispositivo lanza un objeto con una velocidad inicial v_0 (m/s) y un ángulo θ . Se desea saber si el objeto superará un muro de altura H ubicado a una distancia D . El programa debe indicar "Impacta", "Supera" o "No llega".

- **Ayuda: ecuación de un proyectil:**

$$y = D \cdot \tan(\theta) - \frac{g \cdot D^2}{2 \cdot v_0^2 \cdot \cos^2(\theta)}$$

3. Control de Calidad (Estadística)

Problema: Una máquina produce piezas cuyos diámetros se almacenan en un arreglo. Se considera "lote aceptable" si el promedio de los diámetros está dentro de un rango $[min, max]$ y si ninguna pieza individual se desvía más de un 10% del promedio. Se pide, implementar la lógica usando un único ciclo para el promedio y otro para la validación.

4. El Dígito Verificador (Aritmética Modular)

Problema: Un sistema de seguridad usa códigos de 4 dígitos. Para validar un código, se calcula un "Checksum" sumando sus dígitos y aplicando el resto de la división por 7. Si el resultado es igual al último dígito (el 5to), el código es válido. El usuario ingresa un número de 5 dígitos y el programa debe separar el cuerpo del código del dígito verificador y decir si es un código válido.

5. Simulación de Interés Compuesto

Problema: Un usuario deposita un capital C en un banco que paga una tasa mensual r . Cada mes, el usuario retira una cantidad fija W por gastos. para un C y r dados: ¿en cuántos meses el saldo llegará a cero? Obtener un valor de C y r tales que el saldo nunca llega a cero e informar para que valores sucede (el sistema es sostenible).

6. Sensor de Estabilidad (Filtro de Datos)

Problema: Un sensor de presión entrega una lista de N valores. Debido a errores de lectura, se deben ignorar los valores que sean menores a cero (imposibles físicamente) y calcular el promedio de los restantes. Se pide escribir una función que reciba el arreglo y devuelva el promedio "limpio".

7. Optimización de Stock (Logística)

Problema: Una tienda tiene un producto con stock S . Si el stock baja de un punto crítico P , se hace un pedido de reposición. El costo del pedido es: Costo Fijo + (Cantidad * Precio Unitario). Si la cantidad supera las 50 unidades, el Costo Fijo se bonifica (es 0).

Implementar una función que actualice el stock y devuelva el costo del pedido usando **referencias** (&).

8. El Autómata Unidimensional

Problema: Dada una fila de 10 luces que pueden estar encendidas o apagadas; en cada turno, una luz cambia de estado si sus dos vecinos inmediatos son iguales entre sí (ambos encendidos o ambos apagados).

Ayuda: evitar el "efecto dominó" (que el cambio de una celda impacte en ese mismo instante en las celdas contiguas).

9. Geofencing (Punto en Rectángulo)

Problema: Un dron opera en una zona segura definida por un rectángulo con coordenadas (x_1, y_1) y (x_2, y_2) . El programa recibe la ubicación actual (px, py) y debe alertar si el dron está fuera.

- **Requisito:** Crear una struct Punto que contenga x e y.
-

10. Reserva de Butacas (Matrices)

Problema: Un cine tiene una sala de $M \times N$. El arreglo contiene 0 (libre) y 1 (ocupado). Un grupo de K amigos quiere sentarse juntos en la misma fila. El programa debe retornar la fila y la columna inicial donde pueden sentarse.